SQL Query Optimization

EXPLAIN ANALYZE, join algorithms and the N+1 problem

Watch the 4-min video

In one line: find the slow query, read its plan with EXPLAIN ANALYZE, compare estimated and actual rows, fix the scan, join or statistics, and measure again; and kill N+1 with eager loading.

1 Start with the query plan

Your order history page suddenly takes **8 seconds**. The code did not change, but the data grew. Where do you start? With the **query plan**.

Every SQL database turns your query into a plan: which indexes to use, which join algorithms, and in what order. `EXPLAIN` shows the plan the optimizer chose, with **estimated** costs and row counts. `EXPLAIN ANALYZE` actually **runs** the query and adds the **real time and real row counts** for every step.

```
EXPLAIN ANALYZE
SELECT o.id, o.total FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE c.country = 'IN' AND o.status = 'refunded';

Nested Loop  (cost=0.85..412.10 rows=12 width=16) (actual time=0.09..7912.4 rows=43808 loops=1)
  -> Seq Scan on customers c  (rows=3) (actual rows=21904 loops=1)
        Filter: (country = 'IN')
  -> Index Scan using orders_customer_idx on orders o  (rows=4) (actual rows=2 loops=21904)
        Filter: (status = 'refunded')
Execution Time: 7931.2 ms
```

Read it from the innermost step outwards. The key comparison: the optimizer **estimated 3** customers in India but **found 21,904**. A big mismatch like that means it was working from bad information. Believing only 3 outer rows, it chose a nested loop, which then ran the inner lookup 21,904 times.

2 How each table is read



Figure 1. The planner picks a scan from its estimate of how many rows match. A wrong estimate means a wrong scan.

- **Sequential scan:** reads the whole table. Fine for small tables, or when you need most of the rows.
- **Index scan:** jumps through an index to the matching rows. Great when the filter is **selective**.
- **Bitmap scan:** in between. It collects matching row locations from the index, then reads the table pages in physical order.

The key idea is **selectivity**. An index on `status` helps for a rare status, but not for one that matches 90% of rows: thousands of random index-to-row hops cost more than one sequential pass.

3 Join algorithms

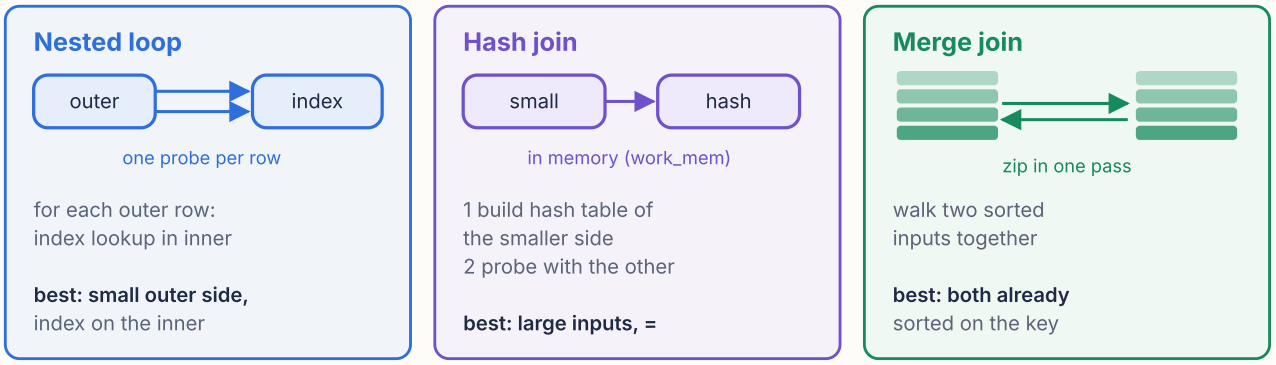


Figure 2. Same result, very different cost. The choice depends on the estimated size of each input.

The optimizer chooses based on **estimated row counts**, and those come from **table statistics** (row counts, value distributions, most common values). **Stale statistics mean bad estimates, and bad plans.** Postgres runs autoanalyze, but after a big data change (a bulk load, a migration, a backfill) run `ANALYZE` yourself.

4 Finding the slow queries first

- **Slow query log:** logs every query slower than a threshold, e.g. `log_min_duration_statement = 500ms` in Postgres, `long_query_time` in MySQL.
- **pg_stat_statements:** a Postgres extension that aggregates every query shape with calls, total time and mean time. Rank by **total time**.

Often the worst offender is not one slow query, but a **fast query that runs thousands of times**. A 2 ms query called 50,000 times a minute costs 100 s of database time every minute, far more than one 3-second report.

5 The N+1 problem

An ORM loads 20 orders with one query. Then, as your code touches each order's items, it **lazily** runs one more query per order: **1 + 20** queries. On a list of 500 orders that is **501 round trips**. Each one is fast, so no single query looks bad in the slow query log, but together the page is slow.

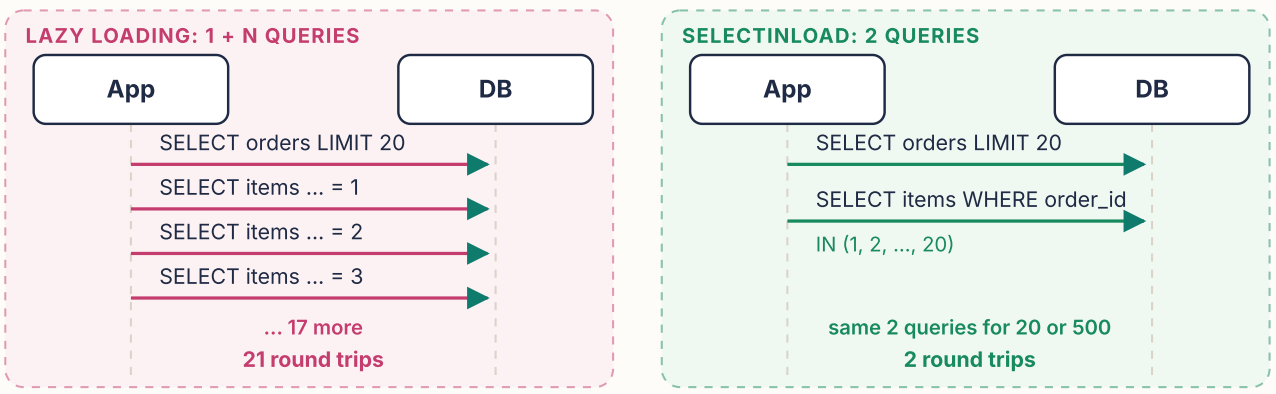


Figure 3. Lazy loading sends one query per parent. Eager loading with selectinload sends one query for the parents and one for all their children.

```
from sqlalchemy import select
from sqlalchemy.orm import selectinload, joinedload, raiseload

# N+1: one query, then one per order when .items is touched
orders = session.scalars(select(Order).limit(500)).all()

# Fix: 2 queries total (orders, then items WHERE order_id IN (...))
orders = session.scalars(
    select(Order).options(selectinload(Order.items)).limit(500)).all()

# Many-to-one: a single JOIN
items = session.scalars(select(Item).options(joinedload(Item.product))).all()

# In development: make any lazy load raise, so N+1 never sneaks back
select(Order).options(raiseload('*'))
```

WORKED EXAMPLE

Order history shows **500 orders**. The app server is **1 ms** round trip from the database and each item query takes **0.5 ms**.

- **Lazy:** 501 queries × (1 + 0.5) ms ≈ **750 ms** of pure waiting, before any rendering.
- **selectinload:** 2 queries; the IN query for all items might take 8 ms. Total ≈ **12 ms**, about **60× faster**.

Use `selectinload` for collections (one-to-many) and `joinedload` for many-to-one; joining a collection would repeat each order's columns once per item.

COMMON MISTAKES

- Optimizing from `EXPLAIN` estimates alone. Use `EXPLAIN ANALYZE` (on a replica or inside a rolled-back transaction for writes: it really runs the query).
- Forgetting `ANALYZE` after a bulk load, then blaming the index.
- Hunting only the single slowest query and missing the fast one called 50,000 times.
- Testing with 10 rows locally: N+1 and sequential scans only hurt at production size.

6 Key terms

Query plan: the steps the optimizer chose

Selectivity: fraction of rows a filter matches

Nested loop / hash / merge join: the three join algorithms

N+1 problem: 1 query for a list plus 1 per item

EXPLAIN ANALYZE: runs the query, shows real times and rows

Statistics: row counts and value distributions the planner uses

pg_stat_statements: Postgres extension ranking query shapes by time

Eager loading: selectinload / joinedload fetch related rows up front

7 Interview questions

Q1. How do you debug a slow query? BASIC

Find it with the slow query log or pg_stat_statements. Run EXPLAIN ANALYZE, compare estimated and actual rows, look for sequential scans on large tables, poor join choices and expensive sorts. Add the right index, rewrite the query, or refresh statistics, then measure again.

Q2. What is the N+1 problem? BASIC

Running one query for a list, then one more for every item, usually from ORM lazy loading. 500 orders means 501 round trips. Fix it with eager loading (selectinload or joinedload) or a single join.

Q3. The plan estimates 3 rows but the actual count is 21,904, and a nested loop takes 8 s. What next? SCENARIO

The statistics are stale or the column distribution is skewed. Run ANALYZE on the table (raise the statistics target for that column if needed) and re-check: with correct estimates the planner will likely pick a hash join and finish in milliseconds.

Q4. No query in the slow log is over 100 ms, yet the database is at 90% CPU. How do you find the cause? SCENARIO

Rank pg_stat_statements by total time, not mean time. Usually it is a cheap query called very often, such as an N+1 lookup at 2 ms × 50,000 calls a minute. Batch it, eager-load it, or cache it.

Q5. An index on status exists but WHERE status = 'completed' uses a sequential scan. Is the planner wrong? SCENARIO

Probably not: if 90% of rows are completed, reading the table sequentially is cheaper than 9 million random index hops. The index helps rare values like 'refunded'; a partial index on those values is even smaller.

Q6. When is a hash join better than a nested loop? TRICKY

When both inputs are large and joined on equality. A nested loop does one index probe per outer row, which is great for 10 outer rows but terrible for 1 million; a hash join reads each side once. It needs memory for the hash table and spills to disk if it does not fit.

Q7. Why prefer selectinload over joinedload for a one-to-many collection? TRICKY

A join repeats every order's columns once per item, so 500 orders with 20 items each send 10,000 wide rows. selectinload sends 500 order rows plus 10,000 narrow item rows in two queries, and it works cleanly with LIMIT on the parent query.

Next: SD-E-008 · ACID Explained backendroadmap.com